Think after basic decoder implementation

Thoughts

alu_sel is ok

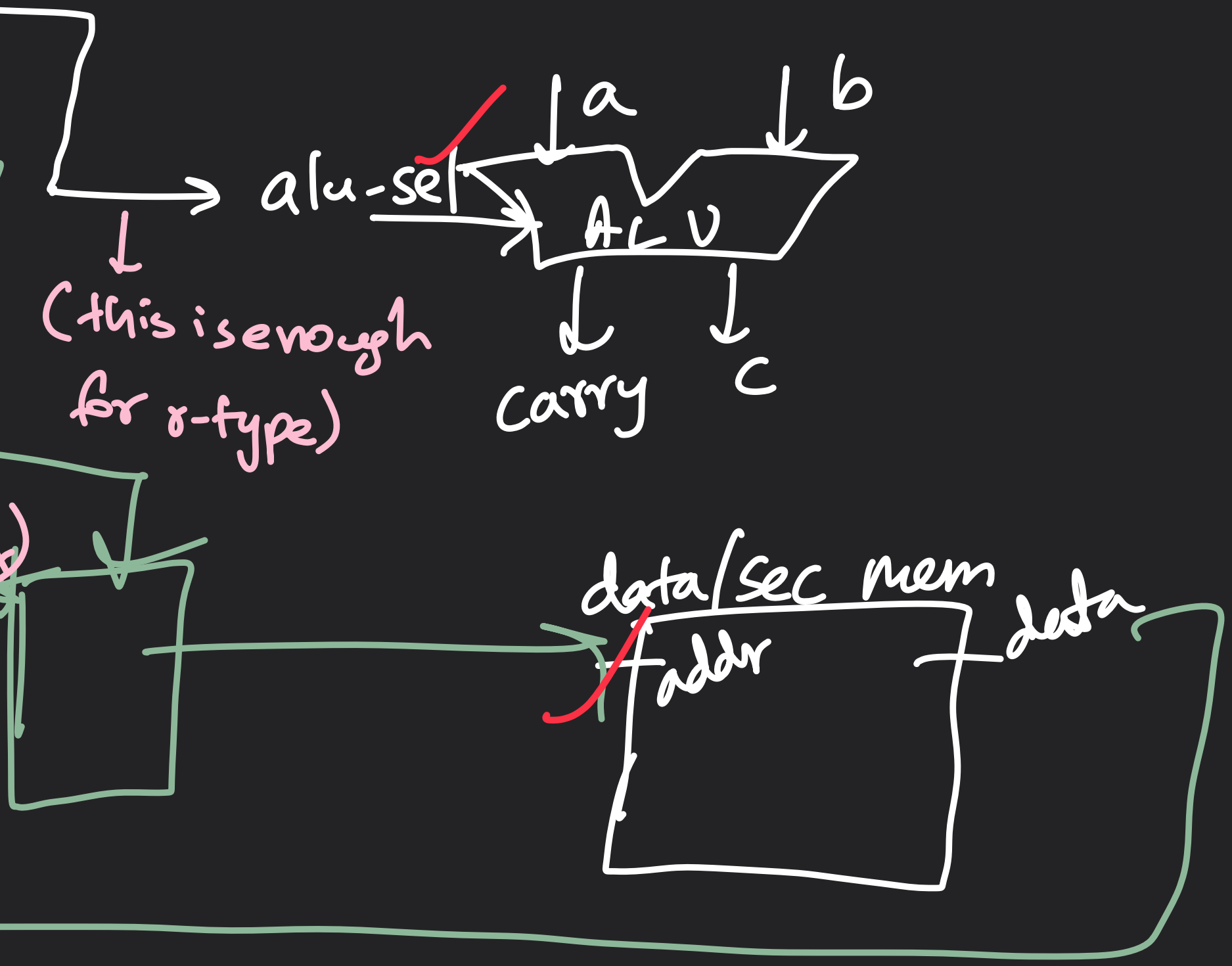
but fetching operands

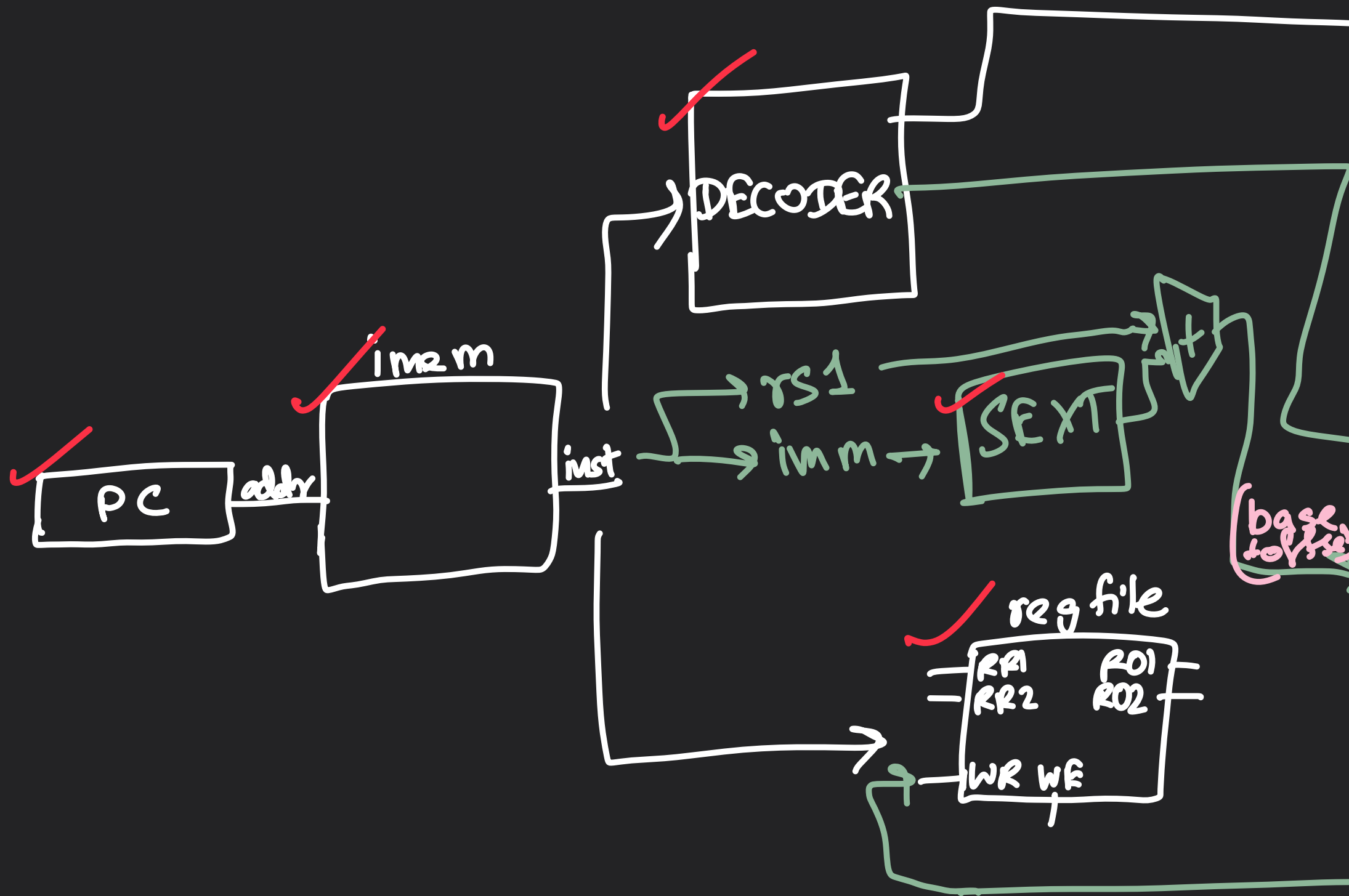
steering that to ALU

writing to register. will that be

to P1 level module

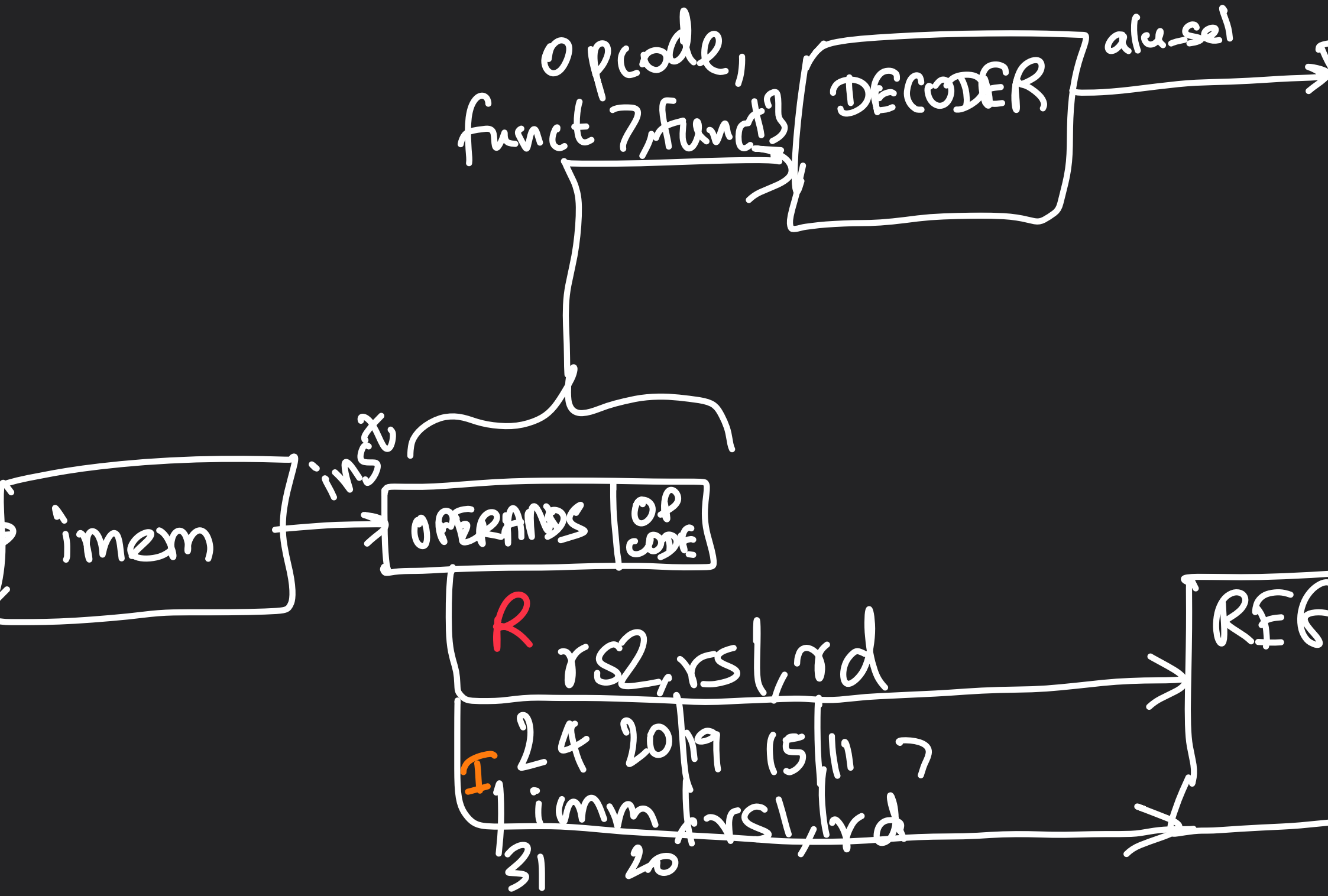
i-type loads


alu-sel
a
b
ALU
carry
c
(this is enough for r-type)
data/sec mem
addr
data


PC
addr
imem
inst
DECODER
rs1
imm
SEXT
+
base
offset
reg file
RR1
RR2
RD1
RD2
WR
WE

b
a
ALU
c

FILE


opcode,
funct7,funct3
DECODER
alu_sel
imem
instr
OPERANDS
OP CODE
R
rs2,rs1,rd
I
24 20
imm
19 15
rs1
11 7
rd
31
20
REG

PC
ありがとう!!

FILE

RANDS | OP CODE

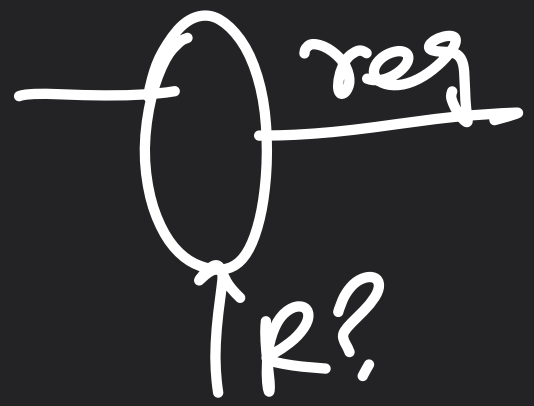
Let's Zoom in here,

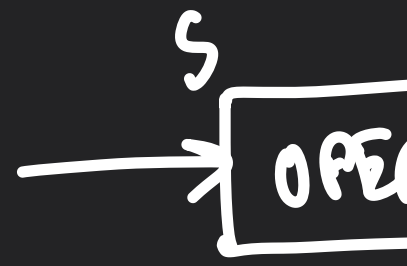
	31	24	20	19	15	11	7
R		RS2		RS1		RD	
I	IMM			RS1		RD	
S	IMM	RS2		RS1		IMM	
B	IMM	RS2		RS1		IMM	
U		IMM				RD	
J		IMM				RD	

RE

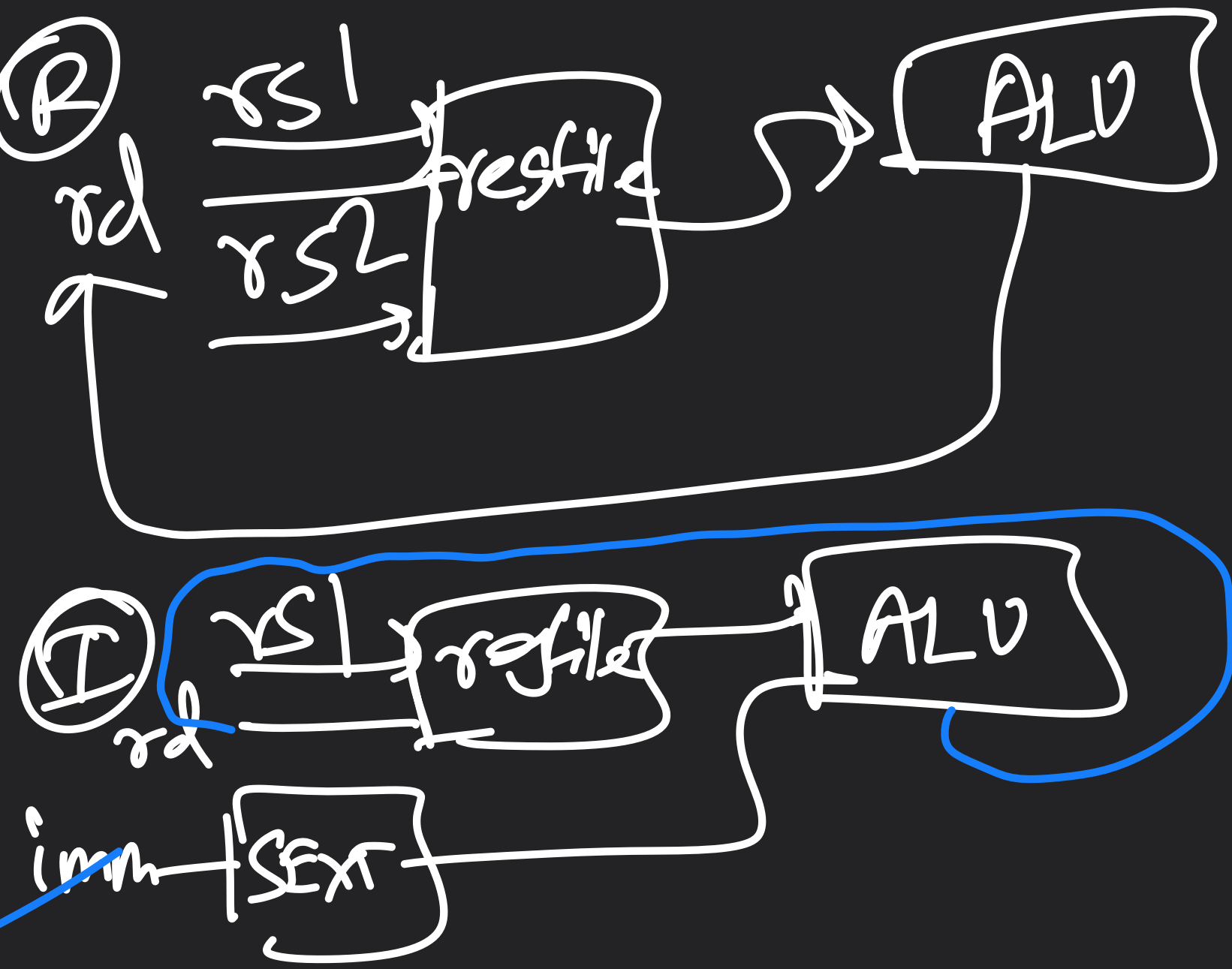
R, I, S, B → RS1

R, I → RS2



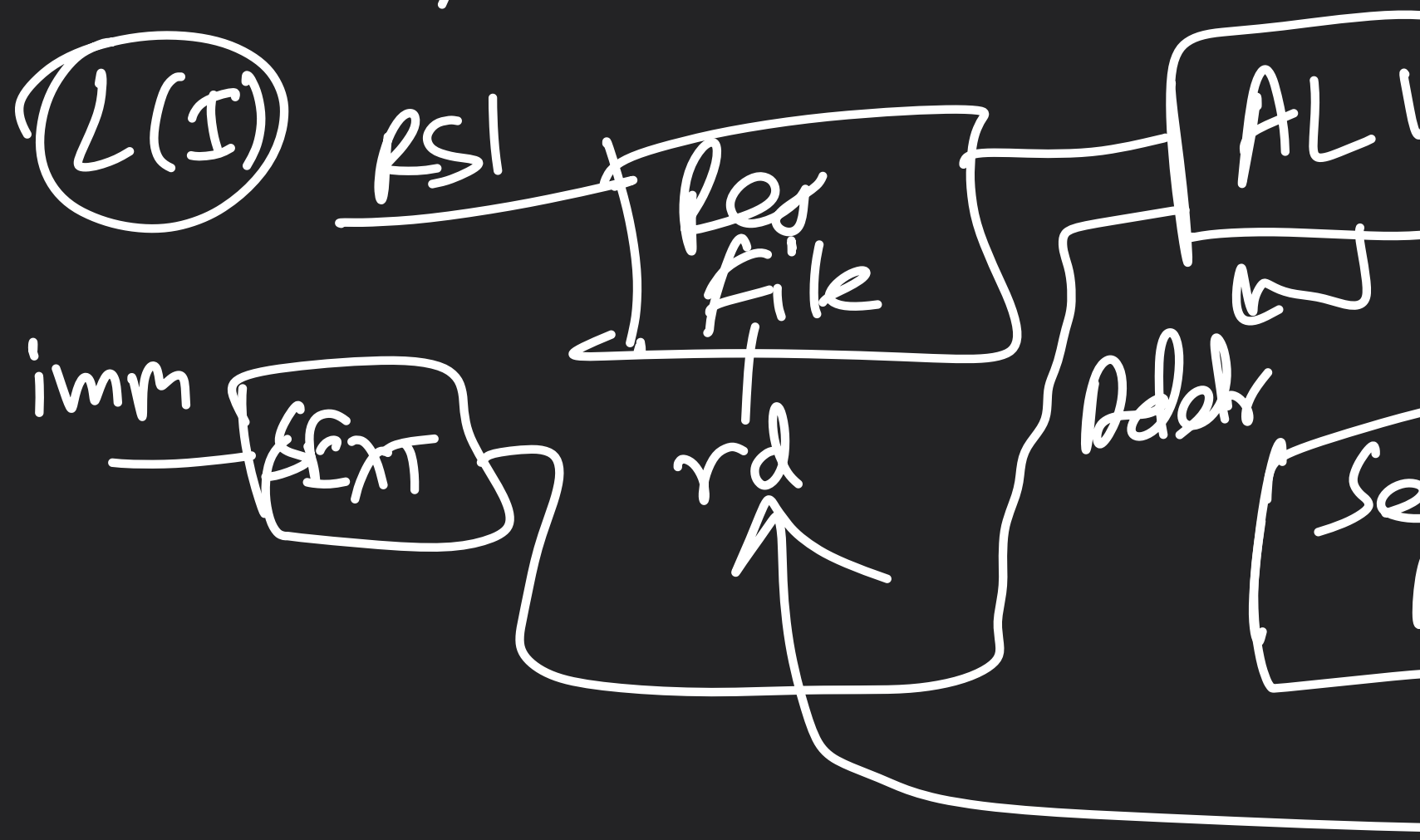


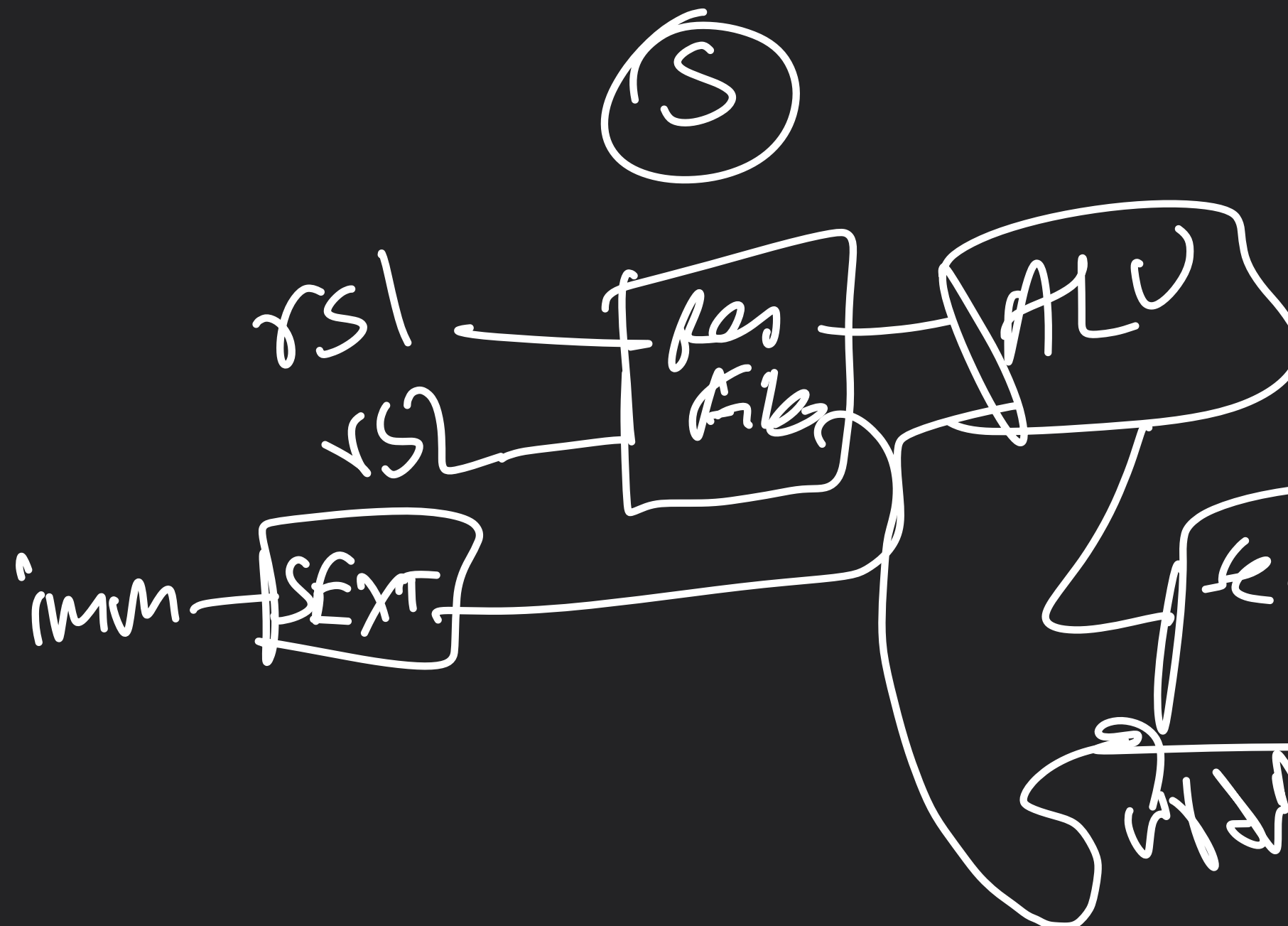
R, I lifecycle:-

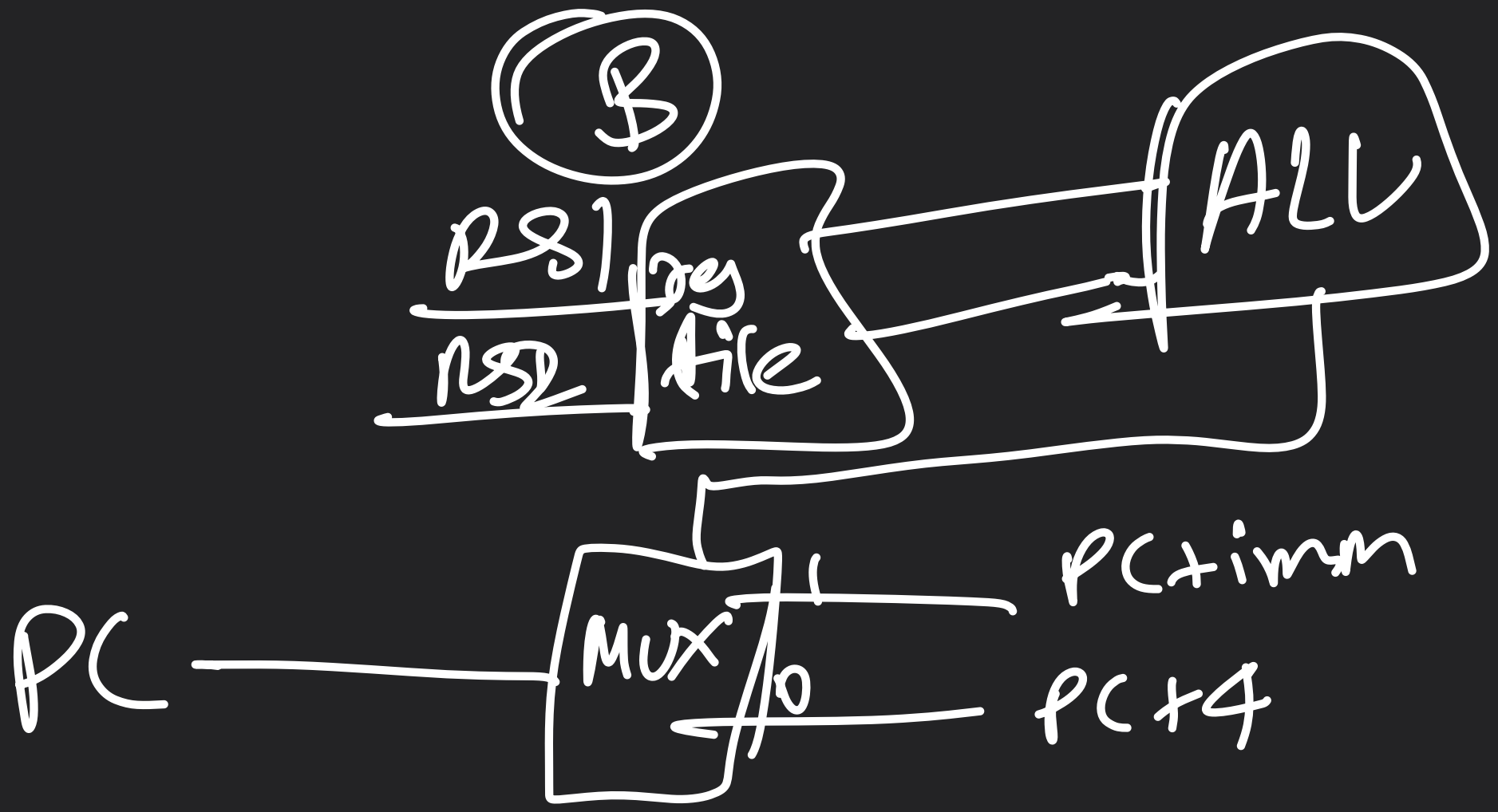


So they only differ in read reg or imm

L(I), S life cycle




S
rs1
rs2
Reg File
ALU
imm
SEXT


B
RS1
RS2
Reg File
ALU
PC
MUX
1
0
PC+imm
PC+4

filtered using instr_type by decoder

B-type , J-type

ONLY JALR

first we will do B type

if (instr_type == 01)

check rs1, rs2 condition

F

PC = PC+4

T

PC = PC+imm

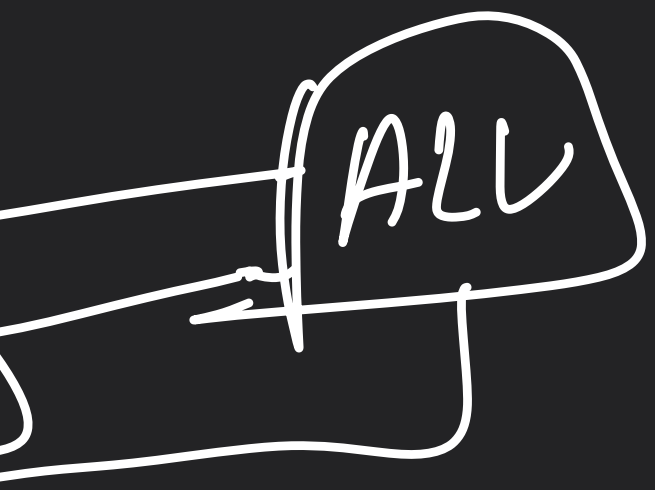
Lets figure out PC first:-

What all insts. play with PC?

ALL B-type, J-type, and one I type

↓

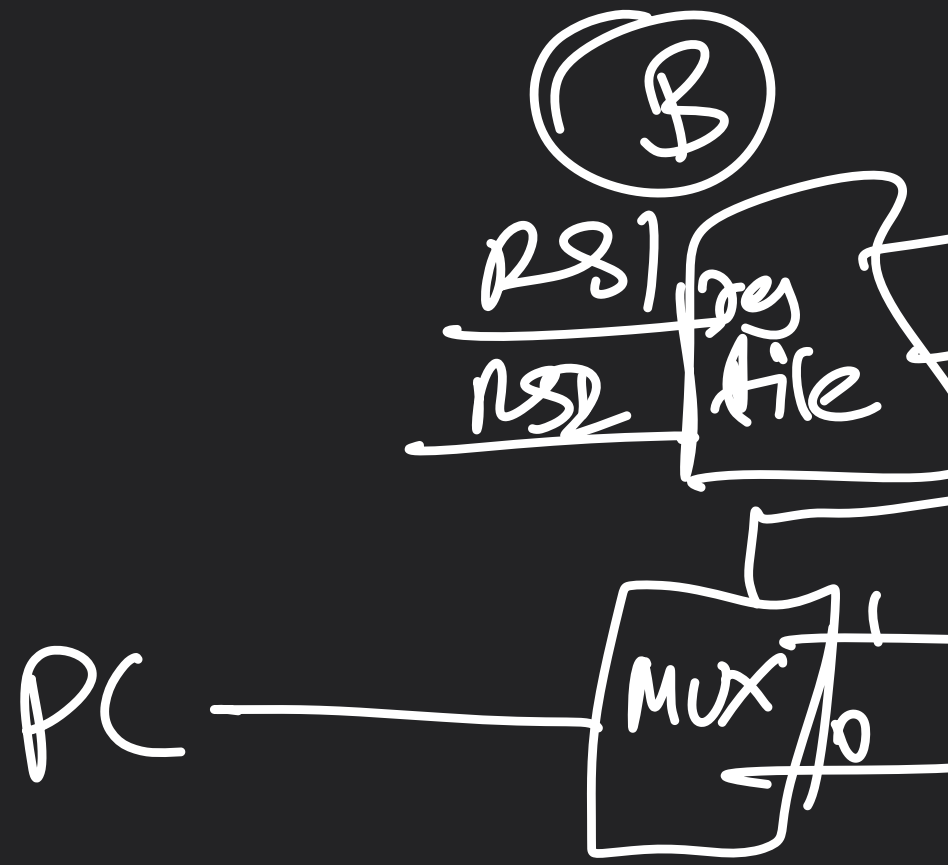
JALR



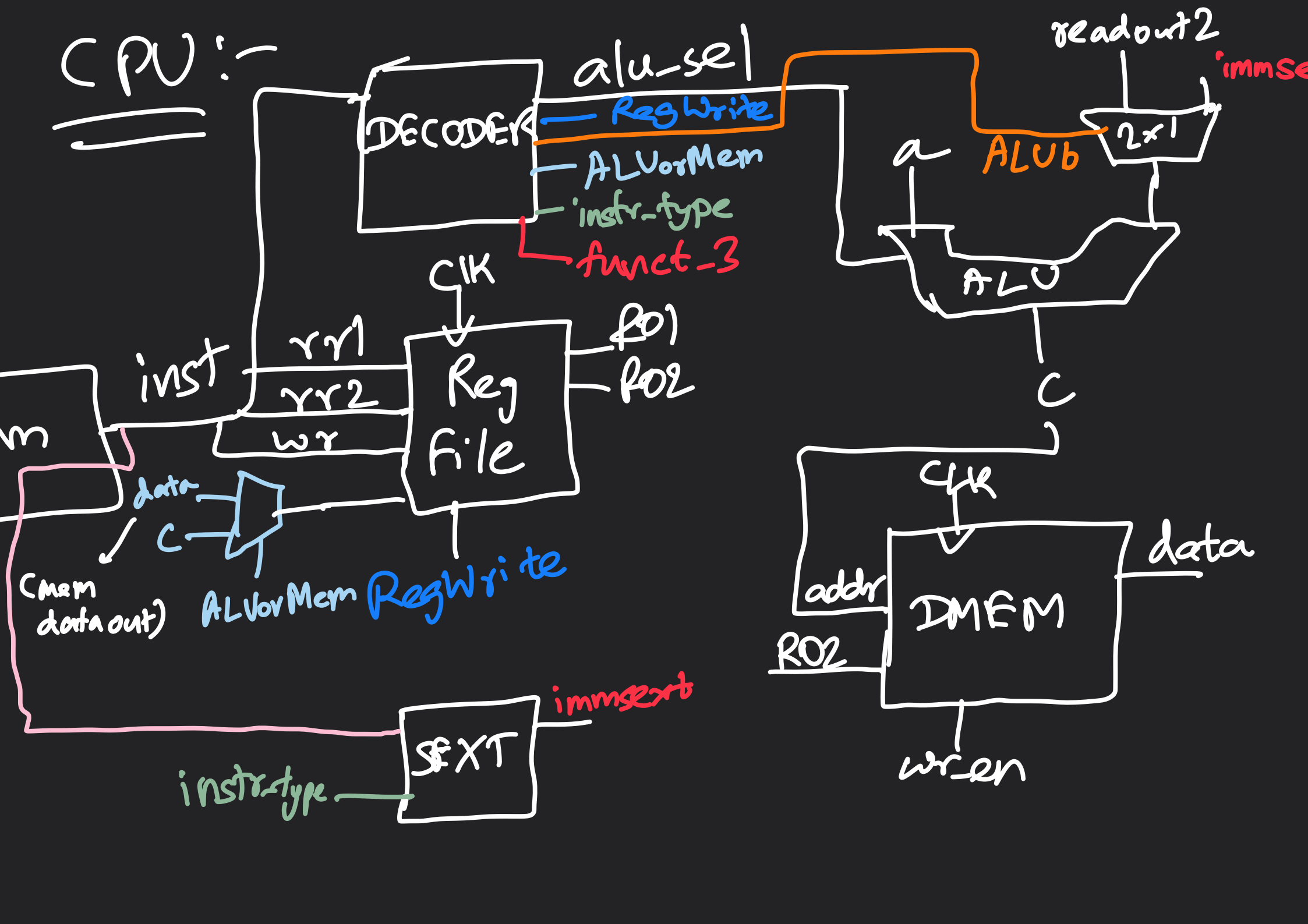
PC+imm

PC+4

ext

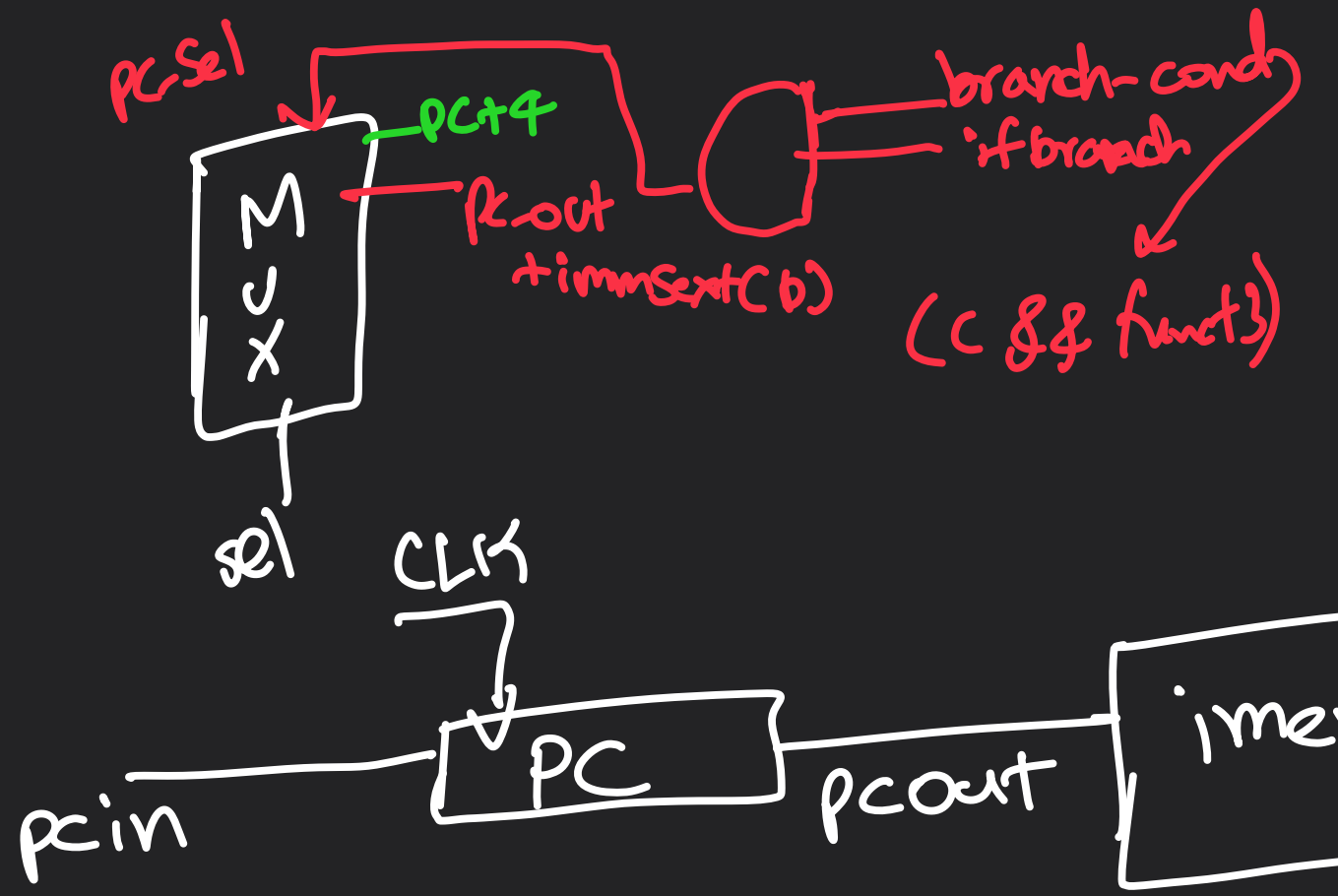


CPU:-


DECODER
alu_sel
RegWrite
ALUorMem
instr_type
funct_3
readout2
immse
2x1
a
ALUb
ALU
c
CLK
Reg File
RO1
RO2
rr1
rr2
wr
inst
m
data
c
(mem dataout)
ALUorMem
RegWrite
CLK
addr
DMEM
data
RO2
wr_en
immsext
SEXT
instr_type

inst.

ed


pc_sel
PC+4
M
U
X
pc_out
+immext(b)
branch_cond
if branch
(c && funct3)
sel
CLK
PC
pcin
pcout
imem

funct_3
will specify
which branch
&& if it is branch
&& cond. satisfie